

Claude Code — Hooks Reference

Complete documentation: what hooks are, where they live, how they are configured, and a full inventory of every active hook on this machine.

Generated: 2026-06-03 • Machine: evw's MacBook Pro • Claude Code v2.1.161

1. What Are Hooks

Hooks are shell commands that Claude Code executes automatically in response to specific lifecycle events — before a tool runs, after a tool completes, or when Claude finishes responding. They run on the **host machine**, outside the Claude model, giving you deterministic control over the agent's behavior.

Hooks can:

- **Block** a tool call before it executes (exit 2 from PreToolUse)
- **Log** every tool call for auditing
- **Auto-format** files after edits
- **Trigger notifications** when Claude finishes
- **Back up work** at session end
- **Suppress or continue** the session (Stop hook with block decision)

2. Untrusted Hooks

Untrusted hooks are hooks defined in project-level settings files (`.claude/settings.json` or `.claude/settings.local.json` inside a project directory) that were not created by the user in the global config.

The distinction matters because:

- **Global hooks** (`~/claude/settings.json`) are set by the owner of the machine and are fully trusted.
- **Project hooks** live inside a repo. If you clone or open a foreign repository, its `.claude/settings.json` could contain malicious hook commands that execute arbitrary shell code on your machine every time Claude runs a tool.

Claude Code surfaces a **prompt before running hooks from an unknown project** so you can review and approve them. A hook is considered "untrusted" until you explicitly approve it for that project. This is the same threat model as `.envrc` files in `direnv` or `package.json` postinstall scripts.

Security note for this machine: The security project's `.claude/settings.local.json` defines only permissions (allow/deny lists), no hooks. All active hooks are defined in the global `~/claude/settings.json` and are therefore fully trusted.

3. Where Hooks Live and How They Are Configured

Configuration files (checked in order)

~/.claude/settings.json	Global (all projects)	Yes	Primary hook config; model, theme, permissions
~/.claude/settings.local.json	Global (machine-local)	Yes	Machine-specific overrides; not committed to git
.claude/settings.json	Project (shared)	Prompt	Committed to repo; reviewed by Claude Code before use
.claude/settings.local.json	Project (local)	Prompt	Not committed; machine-local project overrides

JSON structure

Hooks are defined under the top-level hooks key in any settings file:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",           // tool name to match, or "" for all tools
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/script.sh",
            "timeout": 10,           // optional, seconds
            "statusMessage": "..." // optional, shown in UI while running
          }
        ]
      }
    ]
  }
}
```

How to view your current hooks

Three ways to inspect active hooks:

- 1. **Claude Code UI:** Settings → Hooks (visual editor, shows all active hooks)
- 2. **Shell:** `cat ~/.claude/settings.json | python3 -c "import sys,json; h=json.load(sys.stdin).get('hooks',{}); [print(k,':',len(v),'matchers') for k,v in h.items()]"`
- 3. **Direct file read:** open `~/.claude/settings.json` in any editor — the hooks section is human-readable JSON.

4. All Hook Trigger Events

Claude Code defines the following hook event types:

PreToolUse	Before any tool executes	Yes (exit 2)	tool_name, tool_input, session_id
PostToolUse	After any tool completes	No	tool_name, tool_input, tool_response, session_id

Stop	When Claude finishes a response	Yes (JSON)	transcript_path, session_id
SubagentStop	When a subagent finishes	No	transcript_path, session_id, subagent_id
PreCompact	Before context compaction runs	No	transcript_path, session_id

Hook exit codes

0	Success	Hook ran cleanly; proceed normally
1	Error	Logged as warning; does not block (PostToolUse/Stop)
2	Block	PreToolUse only: tool call is cancelled; stderr shown to Claude
JSON {decision:"block"}	Block stop	Stop hook only: session continues; "reason" fed back as next prompt

Matcher syntax

The matcher field in PreToolUse/PostToolUse filters which tool calls trigger the hook:

- "" (empty string) — matches all tools
- "Bash" — matches only Bash tool calls
- "Write" — matches only Write tool calls
- Multiple matchers: define multiple entries in the array, each with its own matcher

5. Active Hook Inventory (This Machine)

Source: ~/.claude/settings.json • Project hooks: none (project settings.local.json has no hooks key)

PreToolUse — Fires before every tool call

(all tools)	~/.claude/hooks/pre-tool-use.sh	—	Blocks dangerous operations

pre-tool-use.sh — what it blocks:

git ... --no-verify	Blocked — bypasses git hooks
git push --force / -f	Blocked — force-push not allowed
chmod 777 / a+rwX / o+rwX	Blocked — overly permissive
cat/head/tail/vim *.env *.pem *.key	Blocked — shell read of secrets file
cat/head/tail credentials.json	Blocked — credential file read

Read/Write/Edit *.env	Blocked — .env file access
Read/Write/Edit credentials.json	Blocked — credential file access
Read/Write/Edit secrets/**	Blocked — secrets directory access

PostToolUse — Fires after every tool call

(all tools)	~/claude/hooks/post-tool-use.sh	—	Log + auto-format edited files

Auto-format rules (applied on Write/Edit/MultiEdit):

.js .jsx .ts .tsx .json .css .md .html .yaml .yml	prettier --write
.py	ruff format
.go	gofmt -w
.rb	rubocop --auto-correct
.sh	shfmt -w

Log destination: ~/claude/tool-log.jsonl (append-only JSONL, one entry per tool call, includes logged_at timestamp)

Stop — Fires when Claude finishes a response

1	~/claude/record-session.sh	—	Record transcript path to ~/claude/.last_transcript
2	~/claude/hooks/notify-on-stop.sh	—	macOS desktop notification: "Claude has finished."
3	[inline] git add -A + commit + rsync	30s	Session backup: git commit + copy to ~/claude_home/<project>/
4	[inline] tar + shasum + l5-hash-log.txt	30s	Archive project as .tar.gz, append SHA-256 to L5 hash log

Stop hook #3 — Session backup (inline command):

```
PROJECT_DIR="$PWD"; PROJECT_NAME="$(basename "$PROJECT_DIR")"
if [ -d "$PROJECT_DIR/.git" ]; then
  git -C "$PROJECT_DIR" add -A 2>/dev/null
  git -C "$PROJECT_DIR" diff --cached --quiet ||      git -C "$PROJECT_DIR" commit -m "Claude session backup:"
fi
mkdir -p "$HOME/.claude_home/$PROJECT_NAME"
rsync -a --exclude=.git "$PROJECT_DIR/" "$HOME/.claude_home/$PROJECT_NAME/" 2>/dev/null
```

Stop hook #4 — L5 archive + hash (inline command):

```
PROJECT_DIR="$PWD"; PROJECT_NAME="$(basename "$PROJECT_DIR")"
ARCHIVE="$HOME/.claude_home/${PROJECT_NAME}.tar.gz"
tar -czf "$ARCHIVE" -C "$(dirname "$PROJECT_DIR")" "$PROJECT_NAME" 2>/dev/null
```

```
HASH=$(shasum -a 256 "$ARCHIVE" 2>/dev/null | awk '{print $1}')
[ -n "$HASH" ] && echo "$(date -u +%Y-%m-%dT%H:%M:%SZ) $HASH $PROJECT_NAME" >> "$HOME/dev/security/l5-has
```

6. Plugin Hooks (Installed, Project-Scoped)

The following hooks ship with installed plugins in ~/.claude/plugins/. They are **not active globally** — each fires only when the relevant plugin skill is invoked in a session or when the plugin is explicitly enabled for a project.

ralph-loop	Stop	hooks/stop-hook.sh	Blocks session exit to re-feed prompt; implements /loc
security-guidance	PreToolUse	hooks/sg-python.sh	Python interpreter shim for cross-platform hook execu
explanatory-output-style	PreToolUse	hooks-handlers/session-start.sh	Injects output style instructions at session start
learning-output-style	PreToolUse	hooks-handlers/session-start.sh	Injects learning-focused output style at session start
plugin-dev (examples)	—	examples/load-context.sh etc.	Example hooks for plugin developers (not active)

7. Shell Wrapper (Post-Session Export)

This machine's ~/.zshrc wraps the claude command in a shell function that calls ~/.claude/export-conversation.sh after the CLI exits. This is **not** a Claude Code hook — it runs in the shell after Claude exits and has no access to the hook payload API.

```
claude () {
  command claude "$@"
  ~/.claude/export-conversation.sh # runs after CLI exits
}
```

export-conversation.sh reads ~/.claude/.last_transcript (written by record-session.sh Stop hook #1), parses the JSONL transcript, and saves a Markdown export to ~/Documents/claude-exports/YYYY-MM-DD_.md.

8. All Hook-Related Files on This Machine

~/.claude/settings.json	Config	Defines all active hooks (global)
~/.claude/settings.local.json	Config	Machine-local permissions only (no hooks)
~/dev/security/.claude/settings.local.json	Config	Project permissions only (no hooks)
~/.claude/hooks/pre-tool-use.sh	Script	PreToolUse: blocks dangerous tool calls
~/.claude/hooks/post-tool-use.sh	Script	PostToolUse: JSONL logging + auto-format
~/.claude/hooks/notify-on-stop.sh	Script	Stop #2: macOS desktop notification
~/.claude/record-session.sh	Script	Stop #1: records transcript path
~/.claude/export-conversation.sh	Script	Shell wrapper (not a hook): exports transcript to MD

~/.claude/tool-log.jsonl	Log	Append-only tool call audit log (PostToolUse output)
~/.claude/.last_transcript	State	Stores current session transcript path
~/dev/security/l5-hash-log.txt	Log	L5 archive hashes appended by Stop hook #4
~/.claude_home/<project>/	Archive	Per-project rsync backup (Stop hook #3)
~/.claude_home/<project>.tar.gz	Archive	Per-project tar archive (Stop hook #4)

9. Security Considerations

Hooks execute with full user privileges — the same permissions as the running Claude Code process (uid 501, evw). There is no sandbox.

Key risks and mitigations on this machine:

Malicious project hook in cloned repo	All hooks are global-only; no project hooks defined
Hook command injection via tool payload	pre-tool-use.sh validates tool_name + command before exec
Hook leaks secrets via log	post-tool-use.sh logs to local file; tool-log.jsonl not committed
Stop hook auto-commits sensitive files	rsync excludes .git; git add -A could catch .env — watch for this
Hook scripts tampered on disk	scan-hashes.sh hashes all 4 hook scripts per scan
Long-running hook blocks session	Stop hooks #3/#4 have 30s timeout

To audit hook script integrity:

```
bash ~/dev/security/scan-hashes.sh
# Hashes: pre-tool-use.sh, post-tool-use.sh, notify-on-stop.sh,
#         record-session.sh, export-conversation.sh
```